

聚合/组合

继承不是类代码复用的唯一方式

- 在面向对象程序设计中，有些代码复用不宜用继承来实现：
 - 继承除了支持代码复用外，还体现了类之间在概念上的一般与特殊关系（is-a-kind-of）。如果两个类之间没有这个关系，纯粹是为了代码复用而用继承把它们关联起来，这样会造成概念上的混乱。
 - 例如：“飞机”类复用“发动机”类的功能就不宜用继承来实现。

类之间的整体与部分关系

- 类之间除了继承关系外，还存在一种**整体与部分**的关系（is-a-part-of），即一个类的对象包含了另一个类的对象。例如，“飞机”类与“发动机”类之间就是整体与部分的关系。
- 在整体与部分的关系中，新的类（整体类）通过成员类复用已有类的功能。
- 类之间的整体与部分的关系可以是
 - 聚合（aggregation）关系
 - 组合（composition）关系

聚合

- 在聚合关系中，被包含的对象与包含它的对象**独立**创建和消亡，被包含的对象可以脱离包含它的对象独立存在。例如，
 - 一个公司与它的员工之间是聚合关系。
- 实现上，聚合类的成员对象一般是采用**对象指针**表示，用于指向被包含的成员对象，而被包含的**成员对象**是在**外部**创建，然后加入到聚合类对象中来的。

```
class A { ..... };  
class B //B与A是聚合关系  
{ A *pm; //指向成员对象  
public:  
    B(A *p) { pm = p; } //成员对象在聚合类对象外部创建，然后传入  
    ~B() { pm = NULL; } //传进来的成员对象不再是聚合类对象的成员  
    .....  
};  
.....  
A *pa=new A; //创建一个A类对象  
B *pb=new B(pa); //创建一个聚合类对象，其成员对象是pa指向的对象  
.....  
delete pb; //聚合类对象消亡了，其成员对象并没有消亡  
..... // pa指向的对象还可以用在其它地方  
delete pa; //聚合类对象原来的成员对象消亡
```

例：公司由一批员工组成

```
class Employee //职员类
{ string name;
  int salary;
public:
  Employee(const char *s, int n=0):name(s)
  { salary = n; }
  void set_salary(int n) { salary = n; }
  int get_salary() const { return salary; }
  .....
};
```

```
const int MAX_NUM_OF_EMPS=1000;
class Company //公司类
{ String name;
  Employee *group[MAX_NUM_OF_EMPS]; //职员数组
  int num_of_emps; //职员人数
public:
  Company(const char *s):name(s)
  { num_of_emps = 0; }
  ~Company() { num_of_emps = 0; }
  bool add_employee(Employee *e);
  bool remove_employee(Employee *e);
  int get_num_of_emps() { return num_of_emps; }
  .....
};
```

//创建两个公司类对象

```
Company c1("Company_1"),c2("Company_2");
```

//创建两个职员对象

```
Employee e1("Jack",1000),e2("Jane",2000);
```

//职员Jack加入公司Company_1

```
c1.add_employee(&e1);
```

//职员Jane加入公司Company_1

```
c1.add_employee(&e2);
```

//职员Jack从公司Company_1离职

```
c1.remove_employee(&e1);
```

//职员Jack加入公司Company_2

```
c2.add_employee(&e1);
```

.....

组合

- 在组合关系中，被包含的对象**随**包含它的对象**创建和消亡**，被包含的对象不能脱离包含它的对象独立存在。例如，
 - 一个人与他的头、手和脚之间则是组合关系。
- 实现上，组合类的成员对象一般**直接是对象**，有时也可以采用**对象指针**表示，但不管是什么表示形式，成员对象一定是在组合类对象内部创建并随着组合类对象消亡。

```
class A
{
    .....
};
class C //C与A是组合关系
{ A a;
public:
    .....
};
.....
```

C *pc=new C; //创建一个组合类对象，其成员对象在组合类对象内部创建

```
.....
delete pc; //组合类对象与其成员对象都消亡了
```

```
class A
{ .....
};
class C //C与A是组合关系
{ A *pm; //指向成员对象
public:
    C() { pm = new A; } //成员对象随组合类对象在内部创建
    ~C() { delete pm; } //成员对象随组合类对象消亡
    .....
};
.....
C *pc=new C; //创建一个组合类对象，其成员对象在组合类对象内部创建
.....
delete pc; //组合类对象与其成员对象都消亡了
```

- 用指针表示成员对象时，既可以实现组合关系，也可以实现聚合关系，它们的区别在于：
 - 在组合关系中，成员对象是由包含它的对象创建和撤销。
 - 在聚合关系中，成员对象不由包含它的对象创建和撤销。
 - 成员对象的创建和撤销的一般原则：谁创建谁撤销！

例：利用一个线性表类实现一个队列类。

- 线性表由若干元素构成，元素之间有线性的次序关系。

```
class LinearList
{
    .....
public:
    bool insert( int x
    bool remove( int
    int element( int
    int search( int x
    int length( ) cons
};
```

- 除了两个元素外，每个元素都有且仅有一个直接前驱元素和一个直接后继元素；
- 在除外的两个元素中，一个只有一个直接前驱元素，另一个只有一个直接后继元素。
- 可以在任意位置插入、删除元素。

- **队列** (Queue) 是一种特殊的线性表，插入操作在一端，删除操作在另一端。又称**先进先出表** (First In First Out, FIFO)

- Queue的实现1 (组合) :

```
class Queue
{   LinearList list;
    public:
        bool en_queue(int i)
        { return list.insert(i,list.length());
        }
        bool de_queue(int &i)
        { return list.remove(i,1);
        }
};
```

- Queue的实现2（继承）：

```
class Queue: private LinearList
{ public:
    bool en_queue(int x)
    { return insert(x,length());
    }
    bool de_queue(int &x)
    { return remove(x,1);
    }
};
```

- 这里为什么不用public继承？

- 实际上，private继承已经退化成组合了！

继承与聚合/组合的比较

- 继承与封装存在矛盾，聚合/组合与封装则不存在这个矛盾。
- 在基于继承的代码复用中，一个类向外界提供两种接口：
 - public：对象（实例）用户
 - public+protected：派生类用户
- 在基于聚合/组合的代码复用中，一个类对外只需一个接口：public。

- 继承的代码复用功能常常可以用组合来实现。

```
class A
{ .....
public:
    void f();
    void g();
};
```

//继承

```
class B: public A
{ .....
public:
    void h();
    .....
};
```

//组合

```
class B
{ .....
    A a;
public:
    void f() { a.f(); }
    void g() { a.g(); }
    void h();
    .....
};
```

- 继承更容易实现子类型：
 - 在C++中，public继承的派生类往往可以看成是基类的子类型。
 - 在需要基类对象的地方可以用派生类对象去替代。
 - 发给基类对象的消息也能发给派生类对象。
- 具有聚合/组合关系的两个类不具有子类型关系！